

Table of Contents

1	POPolitics – Plan Qualité & Pratiques de développement	1
1.1	1. Objectif	1
1.2	2. Méthodologie de développement	1
1.3	3. Standards de code et conventions	2
1.3.1	3.1 Backend (Python / Django)	2
1.3.2	3.2 Frontend (Next.js / React)	2
1.3.3	3.3 Data & ETL	2
1.4	4. Stratégie de tests	2
1.4.1	4.1 Types de tests	2
1.4.2	4.2 Organisation	3
1.5	5. Processus de déploiement	3
1.5.1	5.1 Environnements	3
1.5.2	5.2 Étapes de déploiement (vue générale)	3
1.6	6. Bonnes pratiques & sécurité	3
1.7	7. Onboarding d'un nouveau membre	3

1 POPolitics – Plan Qualité & Pratiques de développement

1.1 1. Objectif

Définir les **règles de qualité**, les **pratiques de développement** et les **processus de test et de déploiement** pour le projet POPolitics.

1.2 2. Méthodologie de développement

- Utilisation d'une approche **Agile / Kanban** (voir document "Méthodologie et gestion de projet").
- Gestion du code source via **Git** (GitHub ou équivalent) :
 - Branche principale : `main` (ou `master`) toujours stable.
 - Branches de fonctionnalités : `feature/<nom-fonctionnalite>`.
 - Branches de correctifs : `fix/<description-courte>`.

- Règles de merge :
 - Pull request obligatoire.
 - Revue de code par au moins un autre membre pour les changements significatifs.
 - Lancement des tests pertinents avant fusion.
-

1.3 3. Standards de code et conventions

1.3.1 3.1 Backend (Python / Django)

- Respect du style **PEP 8**.
- Utilisation d'un linter (par ex. flake8 ou ruff) et/ou d'un formateur (black).
- Découpage clair entre :
 - vues / endpoints,
 - services métier,
 - accès aux données (ORM / requêtes).

1.3.2 3.2 Frontend (Next.js / React)

- Linting via **ESLint** + formateur automatique (Prettier).
- Composants réutilisables pour les éléments d'UI récurrents (boutons, cartes, graphiques).
- Fichiers organisés par domaines fonctionnels (ex. portal/, profile/).

1.3.3 3.3 Data & ETL

- Scripts ETL clairs, documentés, versionnés.
 - Nommage explicite des pipelines Kestra.
 - Conservation d'un historique minimal des versions de schémas.
-

1.4 4. Stratégie de tests

1.4.1 4.1 Types de tests

- **Tests unitaires :**
 - Fonctions critiques (transformations ETL, calculs d'indicateurs, endpoints sensibles).
- **Tests d'intégration :**
 - Interactions backend ↔ datamarts PostgreSQL.
 - Appels frontend ↔ APIs REST.
- **Tests manuels :**
 - Parcours utilisateurs principaux (portail collectif, page personnelle, authentification).

1.4.2 4.2 Organisation

- Chaque nouvelle fonctionnalité critique doit être accompagnée **d’au moins un test** (unitaire ou d’intégration).
 - Les tests sont exécutés :
 - localement par le développeur,
 - idéalement automatiquement via un pipeline CI (si mis en place).
-

1.5 5. Processus de déploiement

1.5.1 5.1 Environnements

- **Développement** : exécution locale de la **stack complète en Docker** (7–8 conteneurs : Django, Next.js, PostgreSQL/datamarts, base d’auth, Kestra, etc.), orchestrée via `docker-compose`.
- **Démo / pré-prod** : même stack Docker déployée sur une machine/VM partagée, utilisée pour les démonstrations et la maintenance.

1.5.2 5.2 Étapes de déploiement (vue générale)

1. Mise à jour du code sur la branche principale.
2. Construction / mise à jour des images ou services (Django, Next.js, Kestra).
3. Application des migrations de base de données si nécessaire.
4. Vérification rapide des parcours critiques (smoke tests).

Les détails concrets (scripts, commandes) pourront être précisés en fonction de l’infrastructure retenue.

1.6 6. Bonnes pratiques & sécurité

- Ne jamais committer de secrets (mots de passe, tokens) dans le dépôt.
 - Utiliser des **variables d’environnement** et/ou un gestionnaire de secrets.
 - Journaliser les événements importants (erreurs, échecs de jobs Kestra) sans stocker de données sensibles inutiles.
 - Mettre en place des règles minimales de contrôle d’accès :
 - vérification systématique de l’authentification,
 - vérification des droits avant chaque action sensible.
-

1.7 7. Onboarding d’un nouveau membre

Lorsqu’un nouveau membre rejoint le projet :

1. Lecture des documents clés :

- Vision / objectifs du projet.
 - OBS et mapping OBS ↔ WBS.
 - Choix techniques et architecture.
 - Présent document de **Plan Qualité**.
2. Mise en place de l'environnement de développement (suivant un guide rapide à rédiger dans le dépôt technique).
 3. Attribution d'une première tâche simple (correction de bug mineur ou petite fonctionnalité) avec accompagnement.
 4. Revue de code systématique sur ses premières contributions.

Ce plan qualité pourra être détaillé au fur et à mesure de l'avancement du projet et des retours d'expérience de l'équipe.